PATENT APPLICATION

PRELIMINARY SPECIFICATION — DRAFT

Title of Invention:	SYSTEM AND METHOD FOR AUTOMATED, EVIDENCE-BASED VERIFICATION OF SOURCE CODE REVIEW COMMENT RESOLUTION
Applicant:	[TO BE COMPLETED]
Inventor(s):	[TO BE COMPLETED]
Attorney Docket No.:	[TO BE COMPLETED]
Related Applications:	None as of this draft.
Draft date:	25 September 2026

NOTICE: This document is a preliminary technical draft of a patent specification prepared from the applicant's engineering documentation and source-code repository. It is intended solely as a working input for review, revision, and formal preparation by a registered patent attorney or patent agent prior to any filing. It has not been filed with any patent office, does not constitute legal advice, and does not by itself secure any patent right. Claim scope, statutory formalities (including jurisdiction-specific formatting, fee schedules, and inventorship declarations), and patentability (novelty, inventive step, and enablement) have not been assessed by counsel and require professional review before filing.

FIELD OF THE INVENTION

[0001] The present disclosure relates generally to tools for computer-assisted software development, and more particularly to systems and methods that automatically determine, using structural analysis of source code together with rule-based reasoning, whether a modification made to a source code file after a reviewer's comment on that file satisfies the requirement expressed in the comment, and that selectively and safely gate an automated blocking action in dependence on statistically validated performance of the determination.

BACKGROUND OF THE INVENTION

[0002] Collaborative software development commonly employs a code-review workflow in which a proposed change (a “pull request”) is examined by one or more reviewers before it is integrated into a shared codebase. A reviewer may leave a comment on a specific line or region of a file identifying a defect or requesting a change. The author of the pull request then pushes one or more additional commits and marks the resulting discussion thread as “resolved.”

[0003] A significant deficiency of existing source-code hosting platforms and associated tooling is that marking a discussion thread as resolved is a purely administrative act performed by the author, and is not verified against the actual content of the subsequent code changes. A thread may be resolved notwithstanding that the underlying change is absent, incomplete, misplaced relative to the location identified by the reviewer, or consists only of a comment, a log statement, or a docstring that mentions the requested behaviour without implementing it. Because review capacity is limited, such unverified resolutions frequently escape a reviewer's later attention.

[0004] Static analysis tools address a related but distinct problem: they detect instances of predefined defect patterns (for example, a null-pointer dereference or an unused variable) independent of any particular reviewer comment, and therefore cannot determine whether a free-form natural-language request made by a human reviewer has been satisfied.

[0005] Text-similarity and machine-learned semantic-relevance techniques have also been applied to this class of problem, typically by scoring the similarity between the text of a review comment and the text of a subsequent code change. Such techniques suffer from a characteristic failure mode: because they reward lexical or semantic overlap, they assign a high score to a change that merely repeats or paraphrases the reviewer's request — for example, in a comment, a log message, or an unreachable branch — without implementing the requested behaviour, and, conversely, may assign a low score to a materially correct implementation that does not share vocabulary with the request. Empirical evaluation of several such techniques, on datasets described further below, showed a false-acceptance rate (that is, a rate at which a change that does not, in fact, satisfy the request is nonetheless judged to satisfy it) of between

approximately twelve and eighty-eight percent at any decision threshold usable in practice, which is unacceptably high for a determination that may gate integration of a change.

[0006] There exists a need for a system and method that can determine, with a substantially lower false-acceptance rate than similarity-based techniques, whether a code change occurring after a review comment satisfies the requirement expressed in that comment; that can do so by reference to verifiable structural properties of the code rather than to surface-level textual similarity; that treats any machine-learned component as, at most, non-determinative supporting evidence rather than as the basis for the determination; that is resistant to manipulation by adversarial or misleading text appearing within the repository content itself; and that permits an automated blocking action to be enabled only after the determination's performance has been statistically validated on a category-by-category basis, so that integration of otherwise-acceptable changes is not obstructed by an inadequately validated determination.

SUMMARY OF THE INVENTION

[0007] The following summary is provided to introduce a selection of concepts in simplified form. It is not intended to identify all features of the claimed subject matter, nor is it intended to be used to limit the scope of the claimed subject matter, which is set forth in the claims appended hereto.

[0008] In one aspect, a computer-implemented method is provided that receives, from a source code hosting service, data identifying a review comment associated with a pull request, and determines a temporal window comprising the commits made to the affected source code file after the review comment and up to a resolution point, in a manner that accounts for a rebase or force-push affecting the file.

[0009] In another aspect, the method extracts, from the text of the review comment, one or more atomic requirements, each assigned a category selected from a plurality of predefined categories and an ambiguity score, and generates, by parsing the source code file at the end of the temporal window into a syntax tree, a plurality of structural facts (including, without limitation, conditional expressions, function calls, exception handlers, and return values) each associated with a location within the file.

[0010] In another aspect, the method evaluates, using a rule selected according to the category assigned to a given requirement, whether the plurality of structural facts satisfies that requirement, without reliance on a trained machine-learning model for that determination; aggregates the resulting per-requirement statuses into an overall verdict together with a confidence value; and generates an explanation for the verdict that cites the particular structural facts, together with their file location and associated commit identifier, on which the verdict is based.

[0011] In another aspect, a trained machine-learning model configured to estimate a semantic-relevance score between a requirement and the source code is optionally employed, but the resulting score is stored and presented as non-determinative evidence and is, by construction, excluded from the aggregating step, so that the verdict does not depend on the machine-learning model.

[0012] In another aspect, the method treats the text of the review comment, the source code, and any commit message or reply associated with the temporal window exclusively as data rather than as an instruction to the one or more processors performing the method, such that an instruction embedded within such text (for example, an instruction purporting to direct the system to treat a requirement as satisfied) does not alter the verdict.

[0013] In another aspect, a policy module determines a recommended action from the verdict and the confidence value, and permits a blocking action — that is, an action capable of preventing integration of the pull request — only when each of a plurality of independently-configurable conditions is concurrently satisfied. The plurality of conditions comprises: a global enablement setting defaulted to a disabled state; a per-repository operating stage, selected from an ordered plurality of stages, having reached a predetermined minimum stage, wherein a transition to the highest stage additionally requires a minimum elapsed monitoring period and a minimum number of confirmations obtained from human reviewers; a determination that the category of the relevant requirement is statistically eligible, based on a one-sided upper confidence bound, at a predetermined confidence level, of both a false-acceptance rate and a false-blocking rate computed for that category from a labelled evaluation dataset; and a configuration of the source code hosting service requiring the recommended action to be satisfied prior to integration.

[0014] In another aspect, the system receives review-related events from the source code hosting service via a cryptographically signed message delivered to a network endpoint, verifies a signature of the message computed over its unparsed body prior to any parsing of its content, enqueues a corresponding task in a durable, at-most-once job queue, and processes the task using an access token scoped to the single repository and to the minimum permissions needed, thereby re-retrieving the relevant thread and code from the source code hosting service rather than relying on the content of the received message.

[0015] In another aspect, a network-accessible, read-only interface presents the verdict, the explanation, and the source code associated with a given determination, is disabled entirely in the absence of a configured access credential, and removes the stored source code (while retaining the verdict and explanation) after a configurable retention period.

BRIEF DESCRIPTION OF THE DRAWINGS

[0016] FIG. 1 is a block diagram illustrating an exemplary system architecture according to one or more embodiments.

[0017] FIG. 2 is a flow diagram illustrating an exemplary method for verifying whether a code change satisfies a review requirement, according to one or more embodiments.

[0018] FIG. 3 is a block diagram illustrating an exemplary logical arrangement of independently-configurable conditions (“locks”) that govern enablement of an automated blocking action, according to one or more embodiments.

[0019] FIG. 4 is a flow diagram illustrating an exemplary sequence of operations performed in an advisory mode of operation in which the system receives and processes an event from a source code hosting service, according to one or more embodiments.

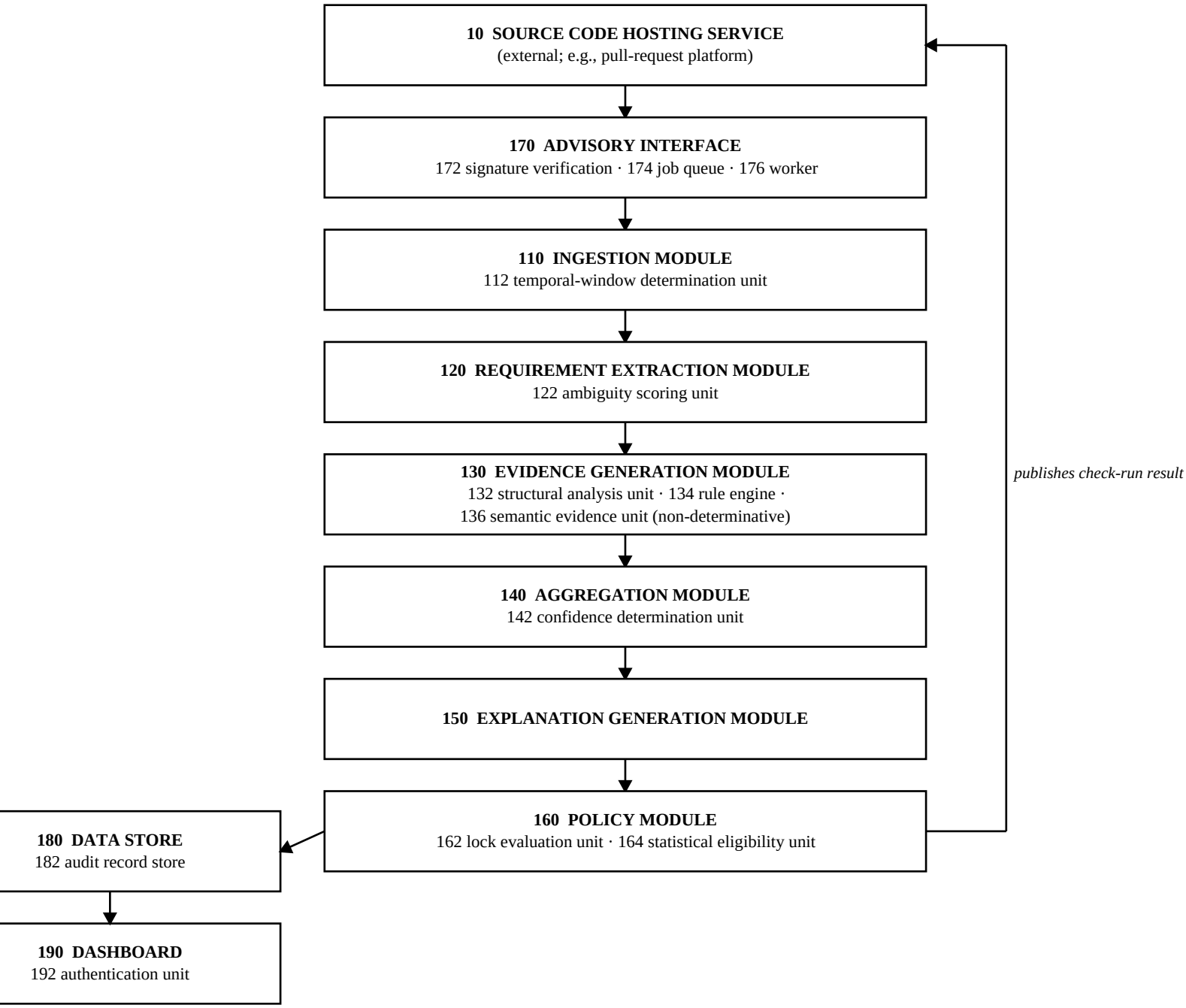
10 SOURCE CODE HOSTING SERVICE
(external; e.g., pull-request platform)
170 ADVISORY INTERFACE
172 signature verification · 174 job queue · 176 worker
110 INGESTION MODULE
112 temporal-window determination unit
120 REQUIREMENT EXTRACTION MODULE
122 ambiguity scoring unit
130 EVIDENCE GENERATION MODULE
132 structural analysis unit · 134 rule engine ·
136 semantic evidence unit (non-determinative)
140 AGGREGATION MODULE
142 confidence determination unit
150 EXPLANATION GENERATION MODULE
160 POLICY MODULE
162 lock evaluation unit · 164 statistical eligibility unit
publishes check-run result
180 DATA STORE
182 audit record store
190 DASHBOARD
192 authentication unit

FIG. 1

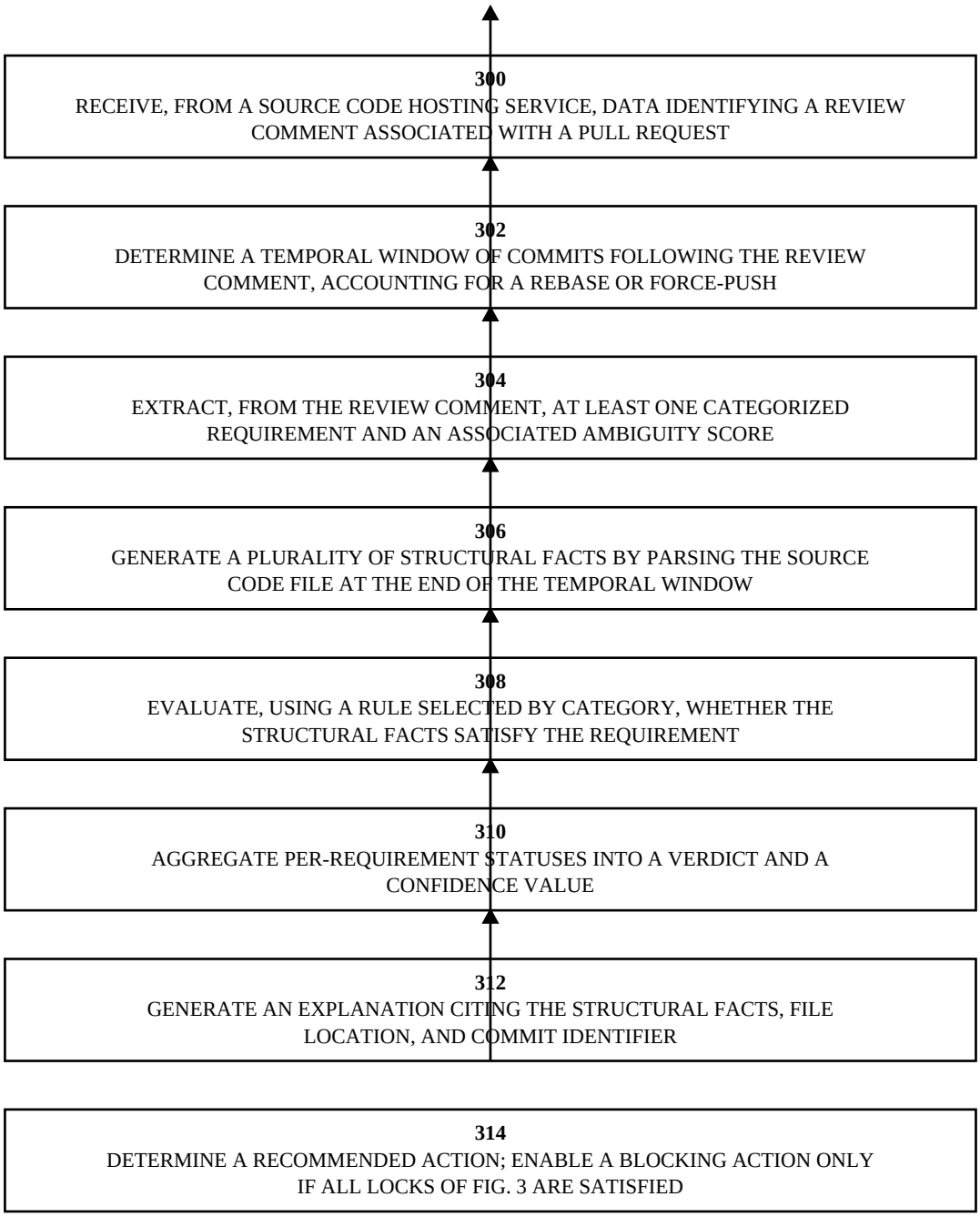
300
RECEIVE, FROM A SOURCE CODE HOSTING SERVICE, DATA IDENTIFYING A REVIEW COMMENT ASSOCIATED WITH A PULL REQUEST
302
DETERMINE A TEMPORAL WINDOW OF COMMITS FOLLOWING THE REVIEW COMMENT, ACCOUNTING FOR A REBASE OR FORCE-PUSH
304
EXTRACT, FROM THE REVIEW COMMENT, AT LEAST ONE CATEGORIZED REQUIREMENT AND AN ASSOCIATED AMBIGUITY SCORE
306
GENERATE A PLURALITY OF STRUCTURAL FACTS BY PARSING THE SOURCE CODE FILE AT THE END OF THE TEMPORAL WINDOW
308
EVALUATE, USING A RULE SELECTED BY CATEGORY, WHETHER THE STRUCTURAL FACTS SATISFY THE REQUIREMENT
310
AGGREGATE PER-REQUIREMENT STATUSES INTO A VERDICT AND A CONFIDENCE VALUE
312
GENERATE AN EXPLANATION CITING THE STRUCTURAL FACTS, FILE LOCATION, AND COMMIT IDENTIFIER
314
DETERMINE A RECOMMENDED ACTION; ENABLE A BLOCKING ACTION ONLY IF ALL LOCKS OF FIG. 3 ARE SATISFIED

FIG. 2

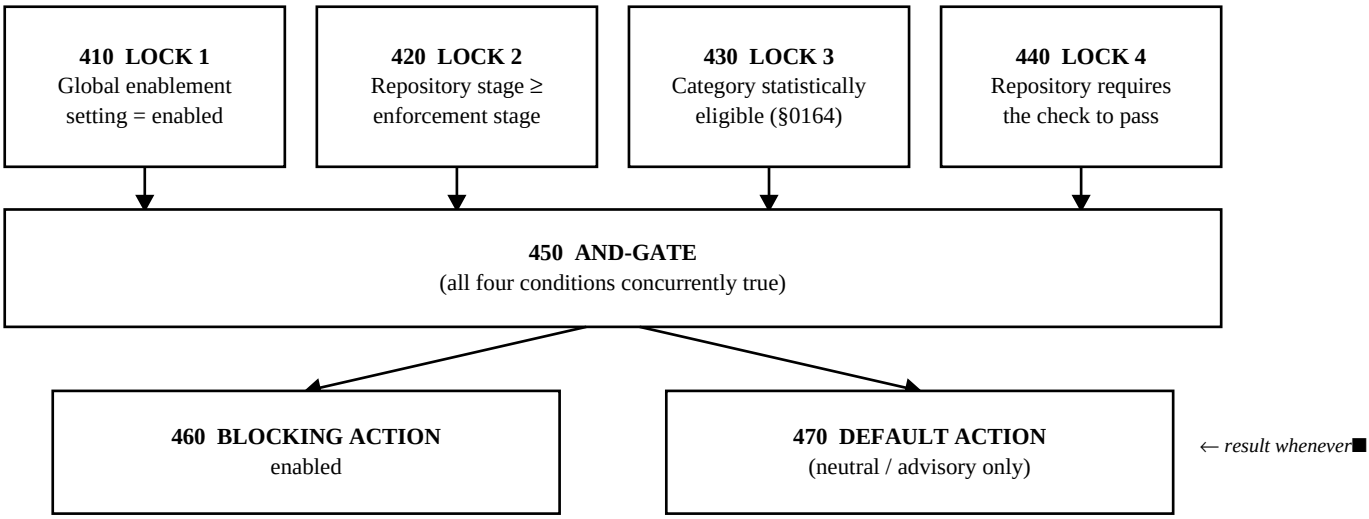
410 LOCK 1
Global enablement setting = enabled
420 LOCK 2
Repository stage ≥ enforcement stage
430 LOCK 3
Category statistically eligible (§0164)
440 LOCK 4
Repository requires the check to pass
450 AND-GATE
(all four conditions concurrently true)
460 BLOCKING ACTION
enabled
470 DEFAULT ACTION
(neutral / advisory only)
← result whenever

FIG. 3

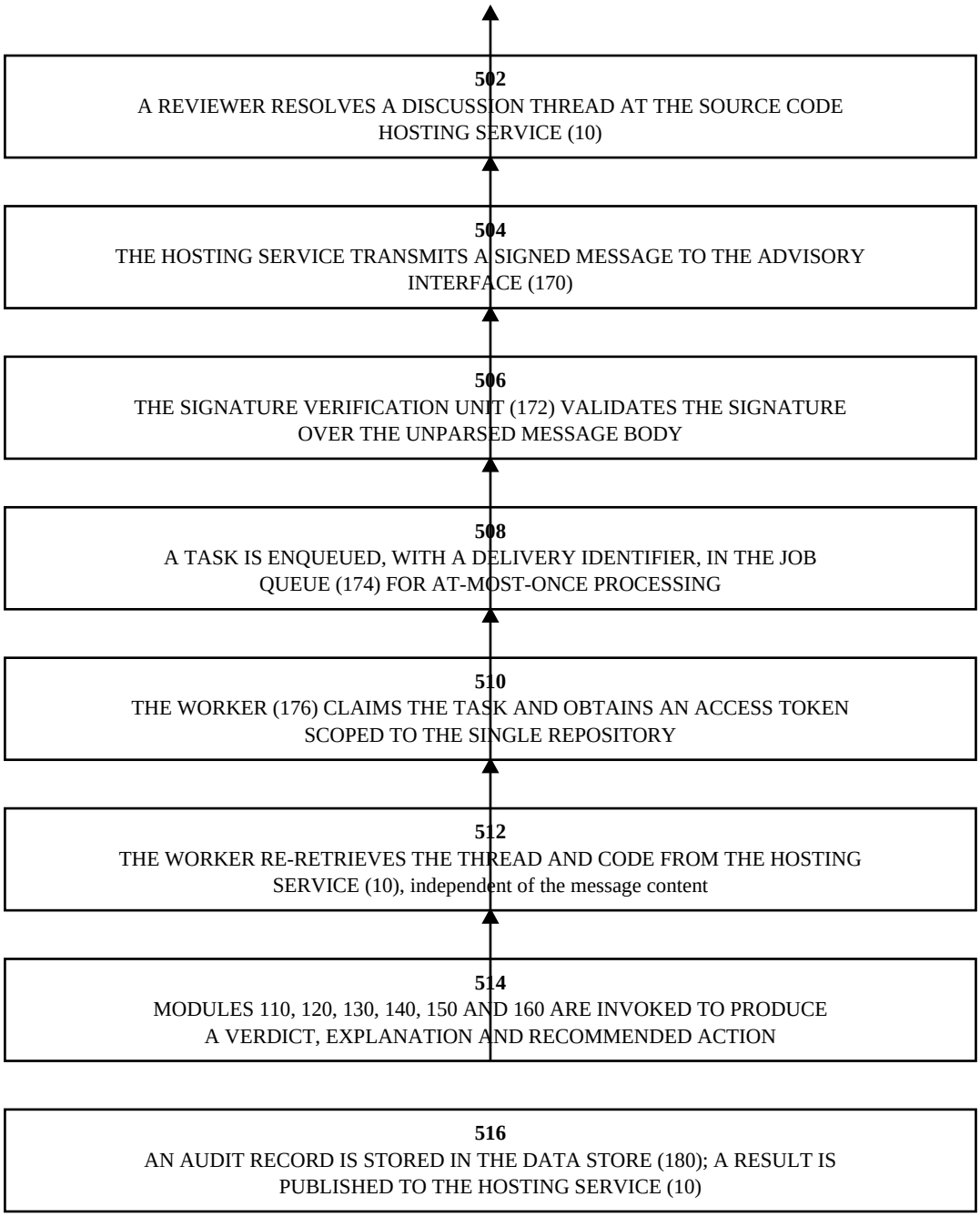
502
A REVIEWER RESOLVES A DISCUSSION THREAD AT THE SOURCE CODE HOSTING SERVICE (10)
504
THE HOSTING SERVICE TRANSMITS A SIGNED MESSAGE TO THE ADVISORY INTERFACE (170)
506
THE SIGNATURE VERIFICATION UNIT (172) VALIDATES THE SIGNATURE OVER THE UNPARSED MESSAGE BODY
508
A TASK IS ENQUEUED, WITH A DELIVERY IDENTIFIER, IN THE JOB QUEUE (174) FOR AT-MOST-ONCE PROCESSING
510
THE WORKER (176) CLAIMS THE TASK AND OBTAINS AN ACCESS TOKEN SCOPED TO THE SINGLE REPOSITORY
512
THE WORKER RE-RETRIEVES THE THREAD AND CODE FROM THE HOSTING SERVICE (10), independent of the message content
514
MODULES 110, 120, 130, 140, 150 AND 160 ARE INVOKED TO PRODUCE A VERDICT, EXPLANATION AND RECOMMENDED ACTION
516
AN AUDIT RECORD IS STORED IN THE DATA STORE (180); A RESULT IS PUBLISHED TO THE HOSTING SERVICE (10)

FIG. 4

DETAILED DESCRIPTION OF THE INVENTION

[0020] The following description sets forth specific details of various embodiments to provide a thorough understanding of the invention. It will be apparent to one skilled in the art, however, that the invention may be practiced without some of these specific details, or with additional elements not described herein. Reference to “one embodiment” or “an embodiment” means that a particular feature is included in at least one embodiment; such references are not necessarily to the same embodiment, and features of different embodiments may be combined. The reference numerals used herein are consistent, where the same feature appears, across FIGS. 1–4, and their use in a description of one embodiment does not limit that feature to the illustrated arrangement.

System Overview (FIG. 1)

[0021] Referring to FIG. 1, a system (100) comprises an advisory interface (170), an ingestion module (110), a requirement extraction module (120), an evidence generation module (130), an aggregation module (140), an explanation generation module (150), a policy module (160), a data store (180) and a dashboard (190), each implemented by one or more processors executing instructions stored in one or more non-transitory computer-readable media, and communicatively coupled to one another and, via a network, to a source code hosting service (10) external to the system.

[0022] The source code hosting service (10) is a computing system, external to the system (100), that hosts one or more source code repositories, supports a pull-request or equivalent code-review workflow, associates review comments with a location in a versioned file, records a resolution state for a discussion thread, and exposes an application-programming interface and a webhook mechanism by which the system (100) may retrieve data and receive event notifications. Nothing herein limits the source code hosting service (10) to any particular commercial platform.

[0023] The advisory interface (170) comprises a signature verification unit (172), a job queue (174), and one or more worker processes (176), and is described in greater detail below in connection with FIG. 4.

Temporal Window Determination (Ingestion Module 110)

[0024] The ingestion module (110) receives, or retrieves via the advisory interface (170), data identifying a review comment, the source code hosting service (10) on which it was made, the repository, the pull request, and the file and location to which the comment is anchored. The ingestion module (110) reconstructs, for the identified comment, a complete discussion thread including the reviewer's comment and any subsequent replies.

[0025] A temporal-window determination unit (112) of the ingestion module (110) determines a temporal window comprising the sequence of commits made to the identified file after creation of the review comment and up to a resolution point associated with the pull request (for example, the head commit of the pull request at the time the thread is marked resolved, or, where a resolution timestamp is unavailable from the source code hosting service (10), the head commit at the time of evaluation). The temporal-window determination unit (112) is configured to correctly identify the beginning and end of the window notwithstanding an intervening rebase or force-push that alters the commit history of the pull request, and to record one or more reliability flags indicating a condition, such as an unresolvable resolution timestamp, under which the accuracy of the window determination or of a subsequent verdict should be discounted.

[0026] The ingestion module (110) outputs a self-contained data structure (referred to herein, without limitation, as a “review case”) comprising the thread, the content of the identified file both before the review comment and at the end of the temporal window, a machine-readable representation of the difference between those two versions, any test file affected within the temporal window, and the reliability flags. Because the review case is self-contained, the remaining modules of the system (100) operate deterministically on it without further recourse to the source code hosting service (10) within a single evaluation.

Requirement Extraction (Module 120)

[0027] The requirement extraction module (120) receives the text of the review comment and produces one or more atomic requirements, each corresponding to a single, checkable action on a single target. Where the comment expresses more than one distinct action (for example, a request to both validate an input and return a particular status code), the requirement extraction module (120) splits the comment into a corresponding plurality of atomic requirements.

[0028] For each requirement, the requirement extraction module (120) assigns a category selected from a plurality of predefined categories, including, in one embodiment, a naming category, a validation category, a testing category, an error-handling category, an application-programming-interface-behaviour category, and a residual, catch-all category encompassing requirements not falling within the foregoing categories (for example, stylistic or refactoring requests, or a reviewer-supplied code suggestion accompanying the comment).

[0029] An ambiguity scoring unit (122) of the requirement extraction module (120) computes an ambiguity score for each requirement, indicative of whether a competent developer could determine, from the comment alone, what specific change is being requested. Where the ambiguity score exceeds a predetermined threshold, the requirement is marked as ambiguous, and, as described below, a status of “uncertain” is assigned to that requirement independent of the outcome of the evidence generation module (130), so that the system (100) does not

generate a false determination for a request that is not, in fact, objectively verifiable.

Evidence Generation (Module 130)

[0030] The evidence generation module (130) comprises a structural analysis unit (132), a rule engine (134), and, optionally, a semantic evidence unit (136).

[0031] The structural analysis unit (132) parses the source code file, at the end of the temporal window, into a syntax tree, and derives therefrom a plurality of structural facts associated with a location within the file. A structural fact comprises, without limitation, at least one of: a conditional expression and its operands; a function or method call and its arguments; a raised or handled exception; a value returned from a function; or a test function and the input values with which it is invoked. The structural analysis unit (132) is further configured to locate, within the file at the end of the temporal window, the function or region corresponding to the location identified by the review comment even where that function has been renamed, relocated within the file, or extracted into a separate helper function relative to the file as it existed before the review comment.

[0032] The rule engine (134) applies, for a given requirement, a rule selected according to the category assigned to that requirement by the requirement extraction module (120), the rule querying the structural facts produced by the structural analysis unit (132) to determine whether the requirement is satisfied. A rule so applied determines a status for the requirement selected from a satisfied status, a not-satisfied status, and an uncertain status, and is configured to return the uncertain status, rather than the satisfied status, whenever the available structural facts are insufficient to make the determination with confidence — for example, where satisfaction depends on the behaviour of a helper function defined in another file, or on a runtime condition not evident from the structural facts. A rule applied to a requirement of the residual, catch-all category is configured, in one embodiment, to evaluate a reviewer-supplied code suggestion by comparing it against the exact code subsequently adopted, and, where no such suggestion is present, to evaluate one or more additional recognized request forms (for example, a request to remove specified code, to add a docstring, or to use one named element in place of another).

[0033] A rule applied by the rule engine (134) further determines that a requirement is satisfied where the structural facts indicate that the requested behaviour was already present in the file before the review comment and remains present at the end of the temporal window, notwithstanding the absence of any change directed to that behaviour within the temporal window; in such a case, a confidence value associated with the requirement is set to a value no higher than a predetermined intermediate level.

[0034] The rule engine (134) does not employ a trained machine-learning model to determine the status of a requirement. This is a deliberate design choice, informed by empirical evaluation (described further below) showing that similarity-based and machine-learned

semantic-relevance techniques exhibit a substantially higher false-acceptance rate than the rule-based determination described herein, because such techniques reward textual or semantic similarity between the requirement and the code rather than verifying the structural presence of the requested behaviour at the requested location.

[0035] The semantic evidence unit (136), where present, computes, using a trained machine-learning model, a relevance score between a requirement and a portion of the code changed within the temporal window, the portion of the code being processed, in one embodiment, with comments and docstrings removed prior to being provided to the model so as to reduce the influence of text that merely repeats the requirement's vocabulary. The relevance score is stored, together with its associated location, as an item of evidence for presentation via the explanation generation module (150) and the dashboard (190), but is, by construction, excluded from the aggregating performed by the aggregation module (140), such that a verdict is identical whether or not the semantic evidence unit (136) is present or its output is included. This non-determinative treatment reflects a further empirical finding that a high relevance score does not reliably indicate that a change is correct — a correct check placed at an incorrect location within the code may nonetheless score highly — whereas a low relevance score does not reliably indicate that a change is incorrect.

Aggregation and Confidence (Module 140)

[0036] The aggregation module (140) combines the per-requirement statuses produced by the evidence generation module (130), together with the ambiguity determination of the requirement extraction module (120), into an overall verdict for the review comment selected from a plurality of predefined verdicts. In one embodiment, the plurality of predefined verdicts comprises: a fully-satisfied verdict, assigned when every requirement associated with the comment has a satisfied status; a partially-satisfied verdict, assigned when at least one requirement has a satisfied status and at least one other requirement has a not-satisfied status; a not-satisfied verdict, assigned when at least one requirement has a not-satisfied status and none has a satisfied status; and an uncertain verdict, assigned when the comment is ambiguous or when a requirement has an uncertain status and none has a not-satisfied status.

[0037] A confidence determination unit (142) of the aggregation module (140) computes a confidence value for the verdict, the confidence value being reduced when a reliability flag recorded by the ingestion module (110) indicates that the temporal window or the resolution state could not be determined with full reliability, and being capped, in one embodiment, below a highest available confidence level absent a predetermined calibration procedure, such that the highest confidence level is not emitted by an uncalibrated configuration of the system (100).

Explanation Generation (Module 150)

[0038] The explanation generation module (150) generates a human-readable explanation of the verdict that cites one or more of the structural facts on which the verdict is based, each citation including an identifier of the file, a line or range of lines within the file, and an identifier of the commit at which the structural fact was observed, so that a human reviewer, or the dashboard (190), may verify the basis for the verdict without re-deriving it. Where the semantic evidence unit (136) is present, the explanation additionally presents its relevance score, labelled as informational and as not affecting the verdict.

Policy and Staged, Statistically-Gated Enforcement (Module 160; FIG. 3)

[0039] The policy module (160) receives the verdict and the confidence value and determines therefrom a recommended action selected from a plurality of predefined actions, comprising, in one embodiment, an allow action, a warning action, a human-review action, and a blocking action, the blocking action being an action capable of preventing integration of the pull request at the source code hosting service (10) (for example, by causing a status check associated with the pull request to indicate failure in a manner the source code hosting service (10) is configured to treat as blocking).

[0040] A lock evaluation unit (162) of the policy module (160) determines the recommended action to be the blocking action only where each of a plurality of independently-configurable conditions, referred to herein as locks and illustrated in FIG. 3, is concurrently satisfied; where any lock is not satisfied, the recommended action defaults to the human-review action or the allow action, as appropriate to the verdict and confidence value, but never to the blocking action. This arrangement ensures that enabling any single lock, by itself, has no effect on whether an integration may be blocked.

[0041] Referring to FIG. 3, the four locks comprise: a first lock (410), being a global enablement setting for the system (100), defaulted to a disabled state; a second lock (420), being a determination that an operating stage maintained for the repository associated with the pull request has reached a predetermined minimum stage among an ordered plurality of stages (for example, an observation stage, an advisory stage, a human-review stage, and an enforcement stage); a third lock (430), being a determination, made by a statistical eligibility unit (164) and described in the following paragraph, that the category of the relevant requirement is statistically eligible; and a fourth lock (440), being a configuration, maintained by or on behalf of the source code hosting service (10) for the repository, that requires the recommended action to be satisfied as a precondition to integration of the pull request. The four locks are combined by an AND-gate (450): the blocking action (460) is enabled only when all four locks are concurrently satisfied; otherwise, a default, non-blocking action (470) results.

[0042] A transition of the operating stage for a repository to a next-higher stage among the ordered plurality of stages is permitted, in one embodiment, only to an adjacent stage (that is,

one stage at a time), is recorded together with an identifier of the actor requesting the transition and a stated justification, and, where the transition is to the enforcement stage, is permitted only after a predetermined minimum period has elapsed since the preceding transition and only after a predetermined minimum number of confirmations have been received from one or more human reviewers, each confirmation being associated with a particular commit of the repository so that a subsequent, unreviewed commit does not inherit an earlier confirmation.

[0043] The statistical eligibility unit (164) computes, from a labelled evaluation dataset partitioned by the plurality of requirement categories, and for a given category, a one-sided upper confidence bound, at a predetermined confidence level (for example, ninety-five percent), of a false-acceptance rate (a rate at which the system determines a not-satisfied case to be satisfied) and, separately, of a false-blocking rate (a rate at which the system determines a satisfied case to be not satisfied), each computed for that category using an exact binomial confidence-interval method. The statistical eligibility unit (164) designates the category as statistically eligible only where both upper confidence bounds do not exceed a predetermined threshold (for example, five percent). Because an upper confidence bound computed from a small number of observed errors, or even from zero observed errors, remains large until a sufficient number of cases has been evaluated, this arrangement has the effect of withholding eligibility for a category until a sufficient volume of validated evaluation data exists for that category, notwithstanding an apparently favourable observed error rate on a smaller dataset.

[0044] The recommended action determined by the policy module (160), together with the verdict, the confidence value, and the explanation, is recorded by the advisory interface (170) as an audit record in the data store (180), and, where the blocking action was not selected, a non-blocking, informational indication (for example, a status check having a neutral disposition, optionally accompanied by a user-actuable control by which a human reviewer may record a confirmation as described above) is published to the source code hosting service (10).

Advisory-Mode Operation and Security (Advisory Interface 170; FIG. 4)

[0045] Referring to FIG. 4, in an advisory mode of operation, a reviewer resolves a discussion thread at the source code hosting service (10) (step 502), causing the source code hosting service (10) to transmit, to the advisory interface (170), a message describing the event, the message being accompanied by a cryptographic signature (step 504).

[0046] The signature verification unit (172) computes a signature over the unparsed body of the received message using a shared secret configured for the system (100), and compares the computed signature with the received signature, rejecting the message where the signatures do not match or where no shared secret is configured, and doing so prior to any parsing of the message body (step 506). This ordering of operations — verifying the signature before parsing the content — reduces the risk that a forged or malformed message could influence the system

(100) other than by being rejected outright.

[0047] Upon successful verification, one or more tasks corresponding to the event are enqueued in the job queue (174), each task being associated with a delivery identifier supplied by the source code hosting service (10) and being enqueued subject to a uniqueness constraint on the delivery identifier, such that a subsequently received, duplicate message bearing the same delivery identifier does not result in duplicate processing of the corresponding task (step 508). A worker process (176) claims an available task from the job queue (174) using a database-level row-locking mechanism that permits a plurality of concurrent worker processes without two such processes claiming the same task, and, upon claiming a task, obtains an access token, scoped to the single repository identified by the task and to a minimum set of permissions sufficient to read the repository's content and to publish a status check, by which to access the source code hosting service (10) (step 510).

[0048] Significantly, the worker process (176) does not rely upon the content of the received message for the substance of the discussion thread or the source code; rather, it re-retrieves the thread and the current code from the source code hosting service (10) using the obtained access token (step 512), and does so, and only thereafter, invokes the ingestion module (110), the requirement extraction module (120), the evidence generation module (130), the aggregation module (140), the explanation generation module (150), and the policy module (160), in sequence, upon the re-retrieved data (step 514). More generally, and independent of the advisory mode of operation specifically, the system (100) is configured to treat the text of the review comment, the source code, and any commit message or thread reply within the temporal window exclusively as data to be analysed rather than as an instruction to be followed, such that text purporting to instruct the system (100) to reach a particular verdict, however phrased or however placed within a comment, string literal, or commit message, does not alter the verdict; this property was verified, in one evaluation, by introducing a plurality of adversarial instructional texts at each of a plurality of locations within a corpus of test cases and confirming that no resulting change in verdict occurred for the rule-based determination.

[0049] Finally, an audit record comprising the re-retrieved review case, the verdict, the confidence value, the explanation, and a cryptographic hash of the inputs used is stored in the data store (180), and a result is published to the source code hosting service (10) via the advisory interface (170) (step 516).

Dashboard (Module 190)

[0050] The dashboard (190) provides a network-accessible, read-only interface through which an operator may view, for a given repository, pull request, or individual determination, the verdict, confidence value, explanation, the source code before and after the temporal window, the computed difference, and a history of prior determinations, operating-stage transitions, and

reviewer confirmations. The dashboard (190) does not expose any control by which the operator may directly alter a verdict, an operating stage, or any other determination made by the system (100); each such alteration, where permitted at all, is made through a separate, audited administrative interface.

[0051] An authentication unit (192) of the dashboard (190) is configured such that the dashboard is entirely disabled, and every request thereto is rejected, in the absence of a configured access credential for the system (100); upon presentation of a valid credential, the authentication unit (192) issues a session token bound, by a cryptographic message-authentication code, to an expiration time and derived from, but not itself disclosing, the configured credential, and applies a rate limit to repeated unsuccessful authentication attempts from a given client. Content originating from the repository (including, without limitation, the review comment, the source code, and the computed difference) is rendered by the dashboard (190) exclusively as non-executable text, irrespective of its content, so that such content cannot cause the execution of an active script within a client rendering the dashboard.

[0052] The data store (180) retains, for each audit record, a stored copy of the source code and review comment associated with the determination for a configurable retention period, after which the stored copy is removed while the verdict, the confidence value, the explanation, and the cryptographic hash of the inputs are retained, so that an audit trail persists after the underlying, potentially proprietary source code has been purged.

Alternative Embodiments

[0053] Although the foregoing description refers, in places, to a particular source code hosting service, a particular programming language, a particular parsing technology, and particular predefined categories, verdicts, actions, and stages, it will be appreciated that the invention is not limited to these particulars. In alternative embodiments: the source code hosting service (10) may be any system supporting an analogous review workflow; the structural analysis unit (132) may parse a programming language other than that used in the examples herein, or may operate upon more than one programming language; the plurality of predefined requirement categories, verdicts, recommended actions, and operating stages may differ in number or definition from those described; the confidence level and error-rate threshold applied by the statistical eligibility unit (164) may be configured to values other than those given by way of example; the semantic evidence unit (136) may be omitted entirely, or may employ any of a variety of machine-learning architectures, without altering its non-determinative role in the aggregating; and the system (100) may be deployed as a single integrated service, as a distributed collection of services, or in part upon infrastructure operated by a third party, without departing from the scope of the invention as set forth in the claims below.

CLAIMS

What is claimed is:

1. A computer-implemented method for verifying whether a code change satisfies a requirement expressed in a source code review comment, the method being performed by one or more processors and comprising:

receiving, from a source code hosting service, data identifying a review comment associated with a pull request, the review comment being anchored to a location within a source code file;

determining a temporal window comprising one or more commits made to the source code file after creation of the review comment and up to a resolution point associated with the pull request, wherein the determining accounts for at least one of a rebase or a force-push affecting the source code file;

extracting, from a text of the review comment, at least one requirement, wherein the extracting comprises assigning a category to the at least one requirement, the category being selected from a plurality of predefined categories, and computing an ambiguity score for the at least one requirement;

generating, by parsing the source code file at an end of the temporal window into a syntax tree, a plurality of structural facts, each structural fact being associated with a location within the source code file and comprising at least one of a conditional expression, a function call, an exception handler, or a return value;

evaluating, using a rule selected according to the category assigned to the at least one requirement, whether the plurality of structural facts satisfies the at least one requirement, thereby determining a status for the at least one requirement, the status being selected from a satisfied status, a not-satisfied status, and an uncertain status;

generating a verdict for the review comment by aggregating the status of the at least one requirement, the verdict being selected from a plurality of predefined verdicts comprising a fully-satisfied verdict, a partially-satisfied verdict, a not-satisfied verdict, and an uncertain verdict;

generating an explanation for the verdict, the explanation citing at least one of the plurality of structural facts together with a file location and a commit identifier associated with the at

least one structural fact; and

determining, based on the verdict and a confidence value associated with the verdict, a recommended action selected from a plurality of predefined actions comprising at least an allow action, a human-review action, and a blocking action,

wherein the determining of the recommended action selects the blocking action only when each of a plurality of independently-configurable conditions is satisfied, the plurality of independently-configurable conditions comprising a global enablement setting, a per-repository operating stage having reached a predetermined minimum stage, a determination that the category assigned to the at least one requirement is statistically eligible based on an upper confidence bound of an error rate computed from a labelled evaluation dataset, and a configuration of the source code hosting service requiring the recommended action to be satisfied prior to integration of the pull request.

2. The method of claim 1, wherein the evaluating of whether the plurality of structural facts satisfies the at least one requirement is performed without use of a trained machine-learning model.

3. The method of claim 1, further comprising computing, using a trained machine-learning model configured to estimate a semantic relevance between the text of the review comment and the source code file, a relevance score, wherein the relevance score is stored as an item of evidence associated with the verdict and is excluded from the aggregating.

4. The method of claim 1, wherein the status of the at least one requirement is set to the uncertain status, independent of the plurality of structural facts, when the ambiguity score exceeds a predetermined threshold.

5. The method of claim 1, wherein the status of the at least one requirement is set to the satisfied status when the plurality of structural facts indicates that the at least one requirement was satisfied prior to creation of the review comment and remains satisfied at the end of the temporal window, and wherein a confidence value determined for that status is capped below a predetermined maximum confidence value.

6. The method of claim 1, further comprising treating textual content of the review comment, the source code file, and a commit message associated with the temporal window exclusively as data for the extracting and the evaluating, such that an instruction embedded within the textual content does not alter the verdict.

7. The method of claim 1, wherein the receiving comprises receiving a signed message from the source code hosting service via a network endpoint, and validating a cryptographic

signature of the signed message, the signature being computed over an unparsed body of the signed message, prior to the extracting.

8. The method of claim 7, further comprising enqueuing, responsive to the receiving, a task in a durable job queue, the task being associated with a delivery identifier, and processing the task at most once notwithstanding a subsequent receipt of a duplicate signed message associated with the same delivery identifier.

9. The method of claim 7, wherein an access token used to access the source code hosting service for a given task is scoped to a single repository identified by the given task and to a read permission and a status-check write permission.

10. The method of claim 1, wherein the determination that the category is statistically eligible comprises computing a one-sided upper confidence bound, at a predetermined confidence level, of a false-acceptance rate and, separately, of a false-blocking rate for the category from the labelled evaluation dataset, and determining the category to be statistically eligible only when both the upper confidence bound of the false-acceptance rate and the upper confidence bound of the false-blocking rate do not exceed a predetermined threshold.

11. The method of claim 1, wherein the per-repository operating stage is one of a plurality of ordered stages, and further comprising restricting a transition of the per-repository operating stage to an adjacent, higher-ordered stage among the plurality of ordered stages, and recording the transition together with an identifier of an actor requesting the transition and a justification for the transition.

12. The method of claim 11, wherein a transition of the per-repository operating stage to a highest-ordered stage among the plurality of ordered stages is permitted only after a predetermined minimum period has elapsed following a preceding transition and only after receipt of a predetermined minimum number of confirmations from one or more human reviewers, each confirmation being associated with a particular commit.

13. The method of claim 1, further comprising generating, for display via a network-accessible interface, a representation of the verdict, the explanation, and a version of the source code file, wherein the representation is generated only responsive to a request accompanied by a valid session credential derived from a configured access credential, and wherein the network-accessible interface is disabled in the absence of the configured access credential.

14. The method of claim 1, further comprising removing a stored copy of the source code file and the review comment associated with the verdict after a predetermined retention period has elapsed, while retaining the verdict, the explanation, and an identifier of the at

least one structural fact.

15. The method of claim 1, wherein the generating of the plurality of structural facts comprises resolving a location, within the source code file at the end of the temporal window, of a function associated with the review comment, even where the function has been renamed, relocated, or extracted into a helper function relative to the source code file at a beginning of the temporal window.

16. A system comprising: one or more processors; and one or more non-transitory computer-readable media storing instructions that, when executed by the one or more processors, cause the system to perform operations comprising the method of claim 1.

17. The system of claim 16, wherein the operations further comprise the operations of any one of claims 2 through 15.

18. One or more non-transitory computer-readable media storing instructions that, when executed by one or more processors, cause the one or more processors to perform the method of claim 1.

19. A computer-implemented method for controlling activation of an automated blocking action in a code-review verification system, the method comprising:

maintaining a global enablement setting, the global enablement setting being defaulted to a disabled state;

maintaining, for each of a plurality of repositories, an operating stage selected from an ordered plurality of operating stages;

computing, from a labelled evaluation dataset partitioned by a plurality of requirement categories, for each requirement category of the plurality of requirement categories, a one-sided upper confidence bound of a false-acceptance rate and a one-sided upper confidence bound of a false-blocking rate;

designating a requirement category of the plurality of requirement categories as eligible only when both the upper confidence bound of the false-acceptance rate and the upper confidence bound of the false-blocking rate for that requirement category do not exceed a predetermined threshold; and

enabling the automated blocking action for a given repository of the plurality of repositories and a given requirement category of the plurality of requirement categories only when: the

global enablement setting is in an enabled state; the operating stage maintained for the given repository is at least a predetermined stage among the ordered plurality of operating stages; the given requirement category is designated as eligible; and a configuration associated with the given repository requires satisfaction of the automated blocking action prior to integration of a pull request.

20. The method of claim 19, wherein the one-sided upper confidence bound of the false-acceptance rate and the one-sided upper confidence bound of the false-blocking rate are each computed using an exact binomial confidence-interval method.

21. The method of claim 19, wherein the operating stage maintained for the given repository transitions to the predetermined stage only after a predetermined minimum period has elapsed following a preceding transition and only after receipt of a predetermined minimum number of confirmations from one or more human reviewers.

22. A system comprising: one or more processors; and one or more non-transitory computer-readable media storing instructions that, when executed by the one or more processors, cause the system to perform the method of claim 19.

ABSTRACT OF THE DISCLOSURE

A computer-implemented system determines whether a source code change made in response to a review comment on a pull request satisfies the requirement expressed in the comment. The system reconstructs a temporal window of commits following the comment, extracts one or more categorized requirements from the comment text, parses the resulting source code into structural facts, and applies a rule selected by category to determine a per-requirement status without relying on a trained machine-learning model, which, when used, is instead retained as non-determinative supporting evidence. Per-requirement statuses are aggregated into an overall verdict together with a confidence value and a citation-based explanation. A policy module maps the verdict and confidence value to a recommended action, enabling an automated blocking action only when a plurality of independently-configurable conditions — including a global switch, a per-repository operating stage, a statistically-validated category eligibility computed from a labelled evaluation dataset, and a repository-level configuration — are concurrently satisfied.